

Study Plan

This section outlines the recommended order to study the topics identified from the provided files (`index.html`, `home.html`, `main.html`, `table.html`, `styling.html`, `css/style.css`, `css/style1.css`). The sequence moves from raw HTML structure and content, into the mechanics of attaching CSS, and finally into visual styling and layout — mirroring how the instructor actually built up the examples.

1. HTML Document Structure & Metadata
 2. Headings, Paragraphs, and Text Flow
 3. Semantic & Inline Text Formatting Tags
 4. HTML Lists
 - Unordered Lists (`<ul>`)
 - Ordered Lists (`<ol>`)
 - Description Lists (`<dl>`)
 5. HTML Media Elements
 - Images (`<img>`)
 - Audio (`<audio>`)
 - Video (`<video>`)
 6. HTML Tables
 - Basic Table Structure & Legacy Attributes
 - Merging Cells with `colspan` and `rowspan`
 7. HTML Forms
 - The `<form>` Container (`method` / `action`)
 - Labels and Basic Inputs
 - Checkboxes vs. Radio Buttons
 - `<textarea>`
 - `<select>`, `<optgroup>`, and `<datalist>`
 8. Attaching CSS to HTML (Inline, Internal, External)
 9. CSS Selectors
 - Universal, Type, Class, and ID Selectors
 - Grouping Selectors
 - Combinators (Descendant, Child, Adjacent Sibling, General Sibling)
 10. CSS Specificity
 11. CSS Color Values (Named, `rgb()`, Hex)
 12. CSS Typography & Text Styling
 13. The CSS Box Model
 14. CSS Backgrounds
 15. The CSS `display` Property
 16. CSS `float` and `clear`
 17. The CSS `position` Property
 18. CSS `transform` (`translate`)
 19. Capstone: Building a Centered Hero Section
-

Deep Explanations and Code Examples

1. HTML Document Structure & Metadata

Every file in this course starts from the same skeleton: an `<html>` root element wrapping a `<head>` and a `<body>`. The `<head>` never renders visible content — it holds *information about* the page (`<title>`, `<meta>`, `<link>`) — while the `<body>` holds everything the user actually sees. In `index.html`, the instructor also demonstrates the `<meta name="keywords">` tag, historically used to feed search engines a comma-separated list of terms describing the page (note the mixed English/Arabic keywords, showing that meta content isn't limited to one language). Modern search engines largely ignore the keywords meta tag for ranking, but it's still taught because it introduces the `<meta>` element's `name/content` attribute pattern, which is reused for far more important tags like `viewport` and `description`.

```
<html>
  <head>
    <title>Web Page</title>
    <meta name="keywords" content="Electro, eletro, ELECTRO, ثمثؤفخ ,
الكترو">
  </head>
  <body>
    <!-- All visible page content goes here -->
  </body>
</html>
```

2. Headings, Paragraphs, and Text Flow

`index.html` and `styling.html` both drill the two most common text containers: headings (`<h1>`–`<h6>`) and paragraphs (`<p>`), plus the generic inline wrapper `<span>`. Headings form a semantic outline of the page — `<h1>` is the most important (ideally one per page) descending in importance to `<h6>`. `<p>` is a **block-level** element (it always starts on a new line and takes the full available width), whereas `<span>` is **inline** (it only takes up as much width as its content and sits in the middle of a line of text). The file also shows an important HTML parsing rule: whitespace collapsing — a `<p>` written across multiple indented lines still renders as a single line of text with normalized spacing, because HTML collapses runs of whitespace/newlines into a single space.

```
<h1>Electro</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>

<p>Paragraph</p>
<p>
  Welcome
  to my website
</p>
```

```
<span>This is a span element</span>
<p>This is a paragraph element</p>
```

3. Semantic & Inline Text Formatting Tags

`styling.html` walks through pairs of tags that look identical by default but carry different *meaning*. `<i>` vs `<em>`: both italicize text, but `<em>` tells assistive technology "this word is emphasized," while `<i>` is purely visual (e.g., a foreign word or a term). `<b>` vs `<strong>` follow the same split — `<strong>` signals real importance (screen readers may change tone), `<b>` is just bold styling. `<mark>` highlights text (like a highlighter pen), `<small>` de-emphasizes text (fine print/disclaimers), `<del>` and `<ins>` mark deleted and inserted content (useful for showing edits, rendered as strikethrough and underline respectively), and `<sup>/<sub>` raise or lower text for exponents and chemical formulas.

```
<i>Hello World</i><br>
<em>Hello World</em><br>
<b>Hello World</b><br>
<strong>Hello World</strong><br>
<mark>Hello World</mark><br>
<small>Hello World</small><br>
<del>Hello World</del><br>
<ins>Hello World</ins><br>
<p>2<sup>2</sup> = 4</p>
<p>H<sub>2</sub>O</p>
```

4. HTML Lists

`index.html` covers all three native list types. **Unordered lists** (`<ul>`) create bulleted lists of `<li>` items; the `type` attribute can control the bullet style (valid values are `disc`, `circle`, or `square` — the file experiments with `type="disc"`, which is not a valid bullet style and would be ignored by the browser, falling back to the default disc). **Ordered lists** (`<ol>`) number their items automatically; `type` switches the numbering style (`a/A` for letters, `i/I` for roman numerals, `1` for digits) and `start` sets the first number/letter in the sequence (`start="5"` with `type="a"` begins at "e"). **Description lists** (`<dl>`) pair terms (`<dt>`) with their definitions (`<dd>`) — ideal for glossaries.

```
<h6>Languages :</h6>
<ul type="disc">
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<ol type="a" start="5">
  <li>Python</li>
  <li>Java</li>
  <li>C++</li>
</ol>
```

```
<dl>
  <dt>HTML</dt>
  <dd>HTML is a markup language used for creating web pages.</dd>
  <dt>CSS</dt>
  <dd>CSS is a style sheet language used for describing the presentation of a
document written in HTML.</dd>
</dl>
```

5. HTML Media Elements

`index.html` introduces the three main embedded-media tags. `<img>` is a **void/self-closing element** requiring `src` (the file path) and, critically, `alt` — fallback text shown if the image fails to load and read aloud by screen readers. `<audio>` and `<video>` are container elements that hold one or more `<source>` children (each with a `src` and MIME `type`), letting the browser pick the first format it supports. Both share playback attributes: `controls` shows the native play/pause UI, `loop` restarts playback at the end, `autoplay` starts playback immediately, and `muted` silences it — note that browsers generally *require* `muted` alongside `autoplay` or they will block autoplay outright. `<video>` additionally supports `poster`, an image shown before the user presses play.

```


<audio controls loop autoplay muted>
  <source src="audio.mp3" type="audio/mpeg" />
</audio>

<video controls loop autoplay muted poster="images/cartoon.jpg">
  <source src="video.mp4" type="video/mp4" />
</video>
```

6. HTML Tables

Basic Structure & Legacy Attributes

`table.html` builds a table from `<table>`, `<tr>` (table row), and `<td>` (table data/cell). The instructor uses several **presentational HTML attributes** — `border`, `bordercolor`, `cellpadding`, `cellspacing`, `bgcolor` — that were common before CSS matured. They still work in most browsers, but modern practice replaces them with CSS (`border`, `padding`, `border-spacing`, `background-color`) so that structure (HTML) and presentation (CSS) stay separated.

Merging Cells with `colspan` and `rowspan`

The second table in `table.html` is a deliberate exercise in cell-merging math. `colspan` merges cells **horizontally** (the instructor's own note calls this "دمج بالعرض" — merging by width), and `rowspan` merges cells **vertically** ("دمج بالطول" — merging by length/height). Tracing the grid: row 1 has a `colspan="2"` cell plus two normal cells = 4 columns. Row 2 starts a `rowspan="3"` cell (occupying rows 2, 3, and 4 in column 1) plus 3 normal cells = 4 columns. Row 3 only needs a `colspan="3"` cell, because column 1 is already filled by the row-2 cell's vertical span — 3 + 1 (spanned) = 4 columns. Row 4 needs exactly 3 normal cells for the same

reason. The key nuance: every row in an HTML table must still resolve to the same total column count once spans are accounted for.

```
<table border="2" bordercolor="red" cellpadding="6">
  <tr>
    <td colspan="2"></td>
    <td></td>
    <td></td>
  </tr>
  <tr>
    <td rowspan="3"></td>
    <td></td>
    <td></td>
    <td></td>
  </tr>
  <tr>
    <td colspan="3"></td>
  </tr>
  <tr>
    <td></td>
    <td></td>
    <td></td>
  </tr>
</table>
```

7. HTML Forms

`main.html` is a single `<form>` that demonstrates nearly every common form control, submitting to `table.html` via a GET request. With `method="get"`, all field values are appended to the `action` URL as a visible query string (fine for searches/filters, unsuitable for sensitive data like passwords); `method="post"` would send the data in the request body instead.

Labels and Basic Inputs

`<label for="password">` is programmatically tied to `<input id="password">` via matching `for/id` values — clicking the label focuses the input, and screen readers announce the label when the input receives focus. The file also shows `type="password"` (masks characters), `type="email"` (adds built-in format validation), `type="number"` with `step="3"` (restricts the spinner increment to jumps of 3), `type="text"` with `required` (blocks submission until filled), and `type="submit"` (a button that submits the form).

```
<label for="password">Enter Password</label>
<input id="password" type="password" placeholder="Enter Password" />

<input type="email" value="Enter Email" />
<input type="number" step="3" />
<input type="text" placeholder="Enter Name" required />
```

Checkboxes vs. Radio Buttons

Checkboxes let a user pick **any number** of independent options — each one toggles on its own, and **checked** pre-selects it. Radio buttons are mutually exclusive: because all of them share the same **name="gender"**, selecting one automatically deselects the others in that group. This shared **name** is what turns a set of separate **<input>** elements into a single logical choice.

```
<h6>Choose Languages</h6>
<label for="html">Html</label>
<input id="html" type="checkbox" checked />
<label for="css">Css</label>
<input id="css" type="checkbox" checked />
<label for="js">Js</label>
<input id="js" type="checkbox" checked />

<h6>Choose Gender</h6>
<label for="male">male</label>
<input id="male" type="radio" name="gender" checked />
<label for="female">female</label>
<input id="female" type="radio" name="gender" />
```

<textarea>

Unlike **<input>**, **<textarea>** is a paired tag (**<textarea>...</textarea>**) built for multi-line text. **cols/rows** set its visible width/height in characters/lines, **placeholder** shows greyed-out hint text, and **minlength/maxlength** constrain how many characters the user can submit.

```
<textarea minlength="5" maxlength="10" placeholder="Enter Message" cols="20"
rows="5"></textarea>
```

<select>, <optgroup>, and <datalist>

<select> creates a closed dropdown of **<option>** elements — the user can only pick from the given list. Marking an option **disabled** **selected** makes it appear as a non-selectable placeholder ("Choose City"). **<optgroup label="...">** visually clusters related options under a heading inside the dropdown. **<datalist>** solves a different problem: it pairs with a **plain text input** via **<input list="cities">**, offering autocomplete suggestions while still letting the user type any free-text value — unlike **<select>**, the input isn't restricted to the listed options.

```
<select>
  <option disabled selected>Choose City</option>
  <optgroup label="city">
    <option value="alex">Alex</option>
    <option value="giza">Giza</option>
    <option value="aswan">Aswan</option>
  </optgroup>
</select>
```

```
<input list="cities">
<datalist id="cities">
  <option value="alex">Alex</option>
  <option value="giza">Giza</option>
  <option value="aswan">Aswan</option>
</datalist>

<button type="submit">Submit</button>
```

8. Attaching CSS to HTML (Inline, Internal, External)

`styling.html` labels a section "INline Css" and, separately, demonstrates **internal CSS** with a `<style>` block in the `<head>`, while its own visible styling actually comes from an **external** stylesheet linked via `<link>`. Together, the files demonstrate all three methods of attaching CSS to HTML, which is foundational before any selector or property makes sense:

- **Inline CSS** — a `style` attribute written directly on one element. Highest priority, but not reusable and mixes content with presentation.
- **Internal CSS** — a `<style>` block inside `<head>`, applying rules to the whole document. Good for single-page demos.
- **External CSS** — a separate `.css` file linked with `<link rel="stylesheet" href="...">`. Reusable across many pages and keeps HTML clean — this is the method used by `home.html` (`css/style1.css`) and `styling.html` (`css/style.css`).

```
<head>
  <link rel="stylesheet" href="css/style.css">
  <style>
    p {
      color: rgb(0, 255, 30);
    }
  </style>
</head>
<body>
  <p style="color: red;">This paragraph uses inline CSS.</p>
</body>
```

9. CSS Selectors

Universal, Type, Class, and ID Selectors

`style.css` isolates each selector type one at a time. A **type selector** (`p`, `h2`) targets every matching tag. A **class selector** (`.new`) targets any element carrying `class="new"` — classes are reusable across many elements. An **ID selector** (`#heading2`) targets exactly one element (IDs must be unique per page). The **universal selector** (`*`) targets every element on the page.

```
#firstParagraph { color: green; }
.second { color: purple; }
```

```
p { color: red; }
.new { color: red; }
#heading2 { color: brown; }
h2 { color: grey; }

* { color: orange; }
```

Grouping Selectors

Placing a comma between selectors applies the same declaration block to all of them at once, avoiding repetition.

```
.new, .second {
    color: blue;
}
h2, h3 {
    color: green;
}
```

Combinators

Using the nested structure from `styling.html`'s `#div1`, the file demonstrates four relationship symbols: a **space** selects any **descendant** at any depth; **>** selects only **direct children**; **+** selects the element immediately **following as a sibling**; and **~** selects **any sibling that comes after**, not just the next one.

```
<div id="div1">
  <p class="second">Paragraph in Div</p>
  <span class="new">Span in Div</span>
  <h2 id="heading2">H2 in Div</h2>
  <p>Welcome to Div</p>
  <div class="heading">
    <p class="new" id="para">Welcome to CSS</p>
  </div>
</div>
```

<code>#div1 p { color: blue; }</code>	<code>/* descendant: every <p> at any depth inside #div1</code>
<code>*/</code>	
<code>#div1 > p { color: red; }</code>	<code>/* child: only <p> elements that are DIRECT</code>
<code>children */</code>	
<code>div + p { color: red; }</code>	<code>/* adjacent sibling: the <p> immediately after a</code>
<code><div> */</code>	
<code>div ~ p { color: blue; }</code>	<code>/* general sibling: any <p> that follows a <div> at</code>
<code>the same level */</code>	

10. CSS Specificity

This is one of the most important nuances in the whole codebase, and `style.css` builds a live example around `styling.html`'s `#div1` block. The instructor's own notes summarize the hierarchy and point values:

```
inline css > id > class > element > *

inline css : 1000
id          : 100
class       : 10
element     : 1
*           : 0
```

Applying this to the file's rules, consider `<p class="new" id="para">Welcome to CSS</p>`. Three separate rules target it:

```
#para { color: orange; }           /* specificity: 100 (one ID) */
.new { color: blue; }             /* specificity: 10  (one class) */
#div1 .heading .new { color: brown; } /* specificity: 120 (1 ID + 2 classes =
100+10+10) */
```

Even though `.new { color: blue; }` appears *last* in the file, `#div1 .heading .new` wins because **specificity is compared before source order** — its combined score (120) beats both `#para` (100) and `.new` (10), so the paragraph renders **brown**. Source order only acts as the tie-breaker when two rules have *equal* specificity.

Similarly, `<p class="second" id="old">Welcome to CSS</p>` is targeted by both `.second { color: red; }` (10) and `#old { color: green; }` (100) — the ID wins, rendering **green**.

11. CSS Color Values (Named, `rgb()`, Hex)

`style.css` sets the same `color` property three times in a row on one rule — a deliberate demonstration that within a single rule, **the last valid declaration wins** (this is the cascade operating at the smallest possible scale). This also showcases the three ways to express a color: a **named/keyword** color (`red`), the **functional** `rgb()` notation (three 0–255 channel values for red, green, blue), and a **hexadecimal** triplet (`#RRGGBB`, base-16). The instructor's own notes explain the hex digit mapping — decimal 0–9 map directly, while 10–15 map to the letters `a–f`, so 255 (the maximum channel value) becomes `ff`.

```
p {
  color: red;           /* overridden */
  color: rgb(200, 150, 255); /* overridden */
  color: #e47723;       /* wins - last declaration */
}
```

Hex digit reference:
0–9 → 0–9

```
10  → a    13  → d
11  → b    14  → e
12  → c    15  → f

255 (max channel value) → ff
```

12. CSS Typography & Text Styling

The active `p` rule at the bottom of `style.css` is a checklist of core text properties: `font-size` (character size), `text-align` (horizontal alignment inside the block), `font-family` (typeface, `monospace` here), `font-style` (`italic/normal`), `font-weight` (boldness, numeric 100–900 or keywords like `bold`), `text-transform` (`lowercase/uppercase/capitalize`, changing *display* only, not the underlying text), `text-indent` (first-line indentation), `line-height` (vertical spacing between lines), `letter-spacing` (space between characters), `word-spacing` (space between words), and `text-decoration` (`overline/underline/line-through`). A key nuance sits inside this exact rule: the `font` shorthand (`font: italic 900 20px arial;`) is declared *after* the individual `font-style`, `font-weight`, `font-size`, and `font-family` properties — because shorthand properties reset every sub-property they cover, this single line **overwrites** all four longhand declarations that came before it.

```
p {
  color: blue;
  font-size: 50px;
  text-align: center;
  font-family: monospace;
  font-style: italic;
  font-weight: 900;
  font: italic 900 20px arial; /* shorthand - overrides the 4 properties above
*/
  text-transform: lowercase;
  text-indent: 70px;
  line-height: 70px;
  letter-spacing: 2px;
  word-spacing: 30px;
  text-decoration: overline;
}
```

13. The CSS Box Model

Every rendered HTML element is a rectangular box made of four layers, from the inside out: **content**, **padding** (space around the content, inside the border), **border** (the visible edge), and **margin** (space outside the border, separating it from other elements). `style.css`'s `#div1` rule exercises the box's dimensions and edge styling directly, and `style1.css` adds shorthand notes for how many values a spacing property can take.

```
#div1 {
  width: 400px;
  height: 400px;
  border: 10px solid orange;
```

```
    outline: 10px solid green;
    border-radius: 50px 10px;
}
.heading {
    border: 10px solid red;
}
```

An important nuance is `outline` vs `border`: both draw a visible line, but `outline` is drawn *outside* the border box and — unlike `border` — never affects the element's layout size or position. `border-radius: 50px 10px` uses two values to round opposite corner pairs (top-left/bottom-right get 50px, top-right/bottom-left get 10px).

The instructor's shorthand notes for margin/padding describe how the number of values changes their meaning:

```
1 value → all four sides
2 values → top/bottom , left/right
3 values → top , left/right , bottom
4 values → top , right , bottom , left    (clockwise)
```

Finally, the notes flag `box-sizing: border-box`, which changes how `width/height` are calculated. By default (`content-box`), `width` only measures the content — padding and border are *added on top*, making the final rendered box larger than the declared width. With `border-box`, padding and border are included *inside* the declared width/height, so the box never grows past the size you set:

```
* {
    box-sizing: border-box; /* width = content + padding + border, capped at the
    declared width */
}
```

14. CSS Backgrounds

`style.css`'s `#div1` example (written, then consolidated into a shorthand) walks through the individual background properties before combining them. `background-image` sets a picture via `url()`; `background-color` shows through anywhere the image doesn't cover (or is used alone with no image); `background-repeat: no-repeat` stops the image from tiling; `background-position` places it within the box (`right bottom` anchors it to the bottom-right corner); and `background-size: cover` scales the image to completely fill the box, cropping if necessary, while preserving its aspect ratio.

```
#div1 {
    background-image: url("../images/cartoon.jpg");
    background-color: rgb(37, 52, 164);
    background-position: right bottom;
    background-repeat: no-repeat;
```

```
background-size: cover;
}
```

These can be combined into the single **background shorthand property**, which the instructor demonstrates directly — note that **background-size** is kept as a separate declaration here, since this particular shorthand syntax combines image, attachment, color, repeat, and position, but not size:

```
#div1 {
  background-size: cover;
  background: url("../images/cartoon.jpg") fixed rgb(37, 52, 164) no-repeat
right bottom;
}
```

15. The CSS **display** Property

style1.css uses **display** to control how an element participates in the page's flow. The three values relevant here: **block** (the element takes the full available width, starts on its own line, and fully respects **width/height/margin/padding** — the default for **div**); **inline** (flows within a line of text, ignores **width/height**, and only takes as much horizontal space as its content — the default for **span** and, notably, **img** unless overridden); and **inline-block** (a hybrid — it flows inline like text, but still respects **width**, **height**, and vertical **margin/padding** like a block). The instructor uses **inline-block** on **.mainDiv** so that multiple fixed-width **divs** can sit side by side, and **display: block** on **#hero-img** to remove the small gap that inline images otherwise leave beneath themselves (a byproduct of inline elements aligning to text baselines).

```
.mainDiv {
  width: 200px;
  display: inline-block; /* sits side-by-side with other divs, but keeps its own
width/height */
}
#hero-img {
  width: 100%;
  height: 100%;
  display: block; /* removes the default inline-image baseline gap */
}
```

16. CSS **float** and **clear**

style1.css briefly touches on the pre-flexbox technique for wrapping layout: **float: left** (or **right**) pulls an element out of the normal document flow and pushes it to one side, allowing surrounding inline content (like text) to wrap around it — the classic "image with text wrapped beside it" effect. Because floated elements are removed from normal flow, a parent containing only floated children can visually collapse to zero height; **clear: both** is applied to a following element to force it to drop *below* any floated elements rather than wrapping beside them, which is the traditional fix for that collapse. While largely superseded today by Flexbox

and Grid for layout, understanding `float/clear` remains useful for reading older codebases and for text-wrap-around-image effects specifically.

```
.mainDiv {  
  float: left;  
}  
.clear {  
  clear: both;  
}
```

17. The CSS `position` Property

`style1.css`'s `.parent`, `.div1`, and `.div2` rules exercise the positioning scheme, which controls how an element is placed relative to the normal flow. `static` is the default (explicitly noted in the file's comment) — `top/left/right/bottom` have no effect. `relative` keeps the element in normal flow (its original space is preserved) but lets `top/left/etc.` nudge it visually from that original spot; critically, it also turns the element into a **positioning context** for any absolutely-positioned descendants. `absolute` removes the element from normal flow entirely and positions it relative to its *nearest positioned ancestor* (the closest parent that isn't `static`) — or relative to the page if no such ancestor exists. `fixed` also removes the element from flow, but anchors it to the *browser viewport*, so it stays in place even while the page scrolls.

```
.parent {  
  position: relative; /* becomes the reference point for absolutely-positioned  
children */  
}  
.div1 {  
  /* default position is static */  
  position: fixed;  
  top: 10px;  
  left: 30px;  
}  
.div2 {  
  background-color: rgb(159, 150, 205);  
  height: 400px;  
}
```

18. CSS `transform (translate)`

`style1.css` uses `transform: translate(x, y)` to shift an element from its current rendered position without affecting the layout of surrounding elements (unlike changing `top/left`, which is layout-based). The key nuance, demonstrated on `.hero-content`, is that **percentage values inside `translate()` are relative to the element's own width/height**, not its parent's — this is what makes the combination below a true centering technique rather than a coincidence.

```
.hero-content {  
  position: absolute;
```

```

    top: 50%; /* moves the element's top-left corner to the vertical
midpoint of the parent */
    left: 50%; /* moves the element's top-left corner to the
horizontal midpoint of the parent */
    transform: translate(-50%, -50%); /* then shifts it back by HALF OF ITS OWN
size, perfectly centering it */
}

```

19. Capstone: Building a Centered Hero Section

`home.html` and `style1.css` combine nearly every layout concept above into one realistic, reusable pattern: a full-width "hero" banner with a background image and centered overlay text — extremely common on real marketing/landing pages. The structure relies on: `position: relative` on the outer `.hero` container to establish a positioning context; `position: absolute` plus the `top: 50% / left: 50% / transform: translate(-50%, -50%)` trick (from Topic 18) to perfectly center `.hero-content` regardless of its own size; `display: block` on the `<img>` to avoid the inline-image gap (Topic 15); and `background-color` plus `text-align: center` on the overlay box for readability against the photo behind it.

```

<div class="hero">
  
  <div class="hero-content">
    <h2>Welcome Hero</h2>
    <p>Lorem ipsum, dolor sit amet consectetur adipisicing elit. Asperiores,
odio laboriosam.</p>
  </div>
</div>

```

```

.hero {
  width: 800px;
  height: 500px;
  position: relative; /* reference point for the overlay below */
}
#hero-img {
  width: 100%;
  height: 100%;
  display: block; /* fills the hero box, no baseline gap */
}
.hero-content {
  background-color: bisque;
  color: grey;
  text-align: center;
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%); /* perfectly centered overlay, any content
size */
}

```

Notes

- **Comments as a teaching tool:** Nearly every file wraps finished/older examples in `<!-- -->` (HTML) or `/* */` (CSS) rather than deleting them, so the instructor could toggle between "before" and "after" states while teaching live. When studying these files directly, expect to un-comment blocks one at a time to see each concept in isolation.
- **A genuine CSS parsing nuance found in `style.css`:** CSS comments **do not nest**. Around line 93, a `/* ... */` comment was opened to hide a block of properties, but a second `/*` appears *inside* it (from an inner background-properties comment) — the parser still closes on the *very next* `*/` it finds, regardless of how many `/*` came before. As a result, the `.heading` and final `p` rules near the bottom of the file end up **active CSS**, not commented out as likely intended. This is genuinely useful to internalize: unlike some languages, CSS comments cannot be safely nested inside one another.
- **Deprecated/legacy attributes appear intentionally:** `bordercolor`, `cellpadding`, `bgcolor` (tables) and presentational HTML in general are shown so students recognize older code, but the modern equivalent is always CSS (`border`, `padding`, `background-color`). When writing new code, prefer the CSS versions.
- **Language note:** Some inline comments in `table.html` are written in Arabic (e.g., "دمج بالعرض" for `colspan`, "دمج بالطول" for `rowspan`) — these have been translated inline above ("merge by width" / "merge by length") for completeness.
- This plan intentionally follows the instructor's own sequence where discoverable (selectors → specificity → colors → typography → box model → backgrounds → display → float → position → transform), since later files (`home.html`) directly depend on positioning and transform concepts introduced earlier in `style.css/style1.css`.